

**Задача 6. (13 т.)** Магазин продава два типа артикули – **бройни** и **количествени**. Една покупка на броен артикул се задава чрез името му, единичната цена и броя закупени артикули, който е цяло число, а една покупка на количествен артикул – чрез името му, цената за килограм и закупеното количество – реално число, което показва колко килограма са закупени.

а) Да се реализира абстрактен базов клас, който обединява общата функционалност за двата вида покупки и има виртуален метод за пресмятане на стойност на една покупка. Да се реализират два производни класа, представящи покупките на двата типа артикули.

б) Да се реализира клас *касова\_бележка*. Бележката трябва да има номер и масив от покупки. В класа за касова бележка да има метод, който пресмята общата стойност на всички покупки от масива.

в) Демонстрирайте използването на класовете в подходяща кратка програма.

**Задача 2.** (15т.) Реализирайте абстрактен базов клас *множество от числа от тип int*, който има метод за проверка на принадлежността към множеството.

1.1. Реализирайте производен клас, който представлява множеството от всички числа, които се делят без остатък на някакво предварително зададено в конструктора число.

1.2. Реализирайте и друг производен клас, който представя множество от числа чрез динамичен масив (който също се задава в конструктора).

1.3. Реализирайте външна за тези класове функция, която по зададен масив от множества и някакво число проверява дали то се съдържа в обединението на множествата от масива.

1.4. Демонстрирайте използването на тази функция в подходяща кратка програма.

**Задача 3.** (10 т.) Да се реализира абстрактен базов клас, представящ обект от файловата система. Този обект се характеризира с име. Класът да предоставя метод за определяне на заеманото от обекта дисково пространство.

- Да се реализира наследник на този клас представящ файл. Файлът трябва да пази информация за размера си.
- Да се реализира наследник на този клас представящ директория. Директорията съхранява масив от обекти от файловата система.
- Напишете кратка програма, която да създаде директория, съдържаща поне една поддиректория и поне един файл и след това да изведе на екрана размера на тази директория.

**Задача 4.** (8т.) Попълнете в празните полета текстът, който се отпечатва на конзолата в резултат на изпълнението на съответните програмни конструкции.

```
#include <iostream.h>

class Base
{
public:
    Base () {cout << "Base::Base()\n"; f(); g();}
    Base (Base&) {cout << "Base::Base(Base&)\n"; f();g();}

    virtual void f () {cout << "Base::f()\n";g();}
    void g () {cout << "Base::g()\n";}

};

class Derived1 : public Base
{
public:
    Derived1 () {cout << "Derived1::Derived1()\n"; f(); g();}
    Derived1 (Derived1 &) {cout <<
"Derived1::Derived1(Derived1&)\n";f();g();}

    void f () {cout << "Derived1::f()\n";g();}
    virtual void g () {cout << "Derived1::g()\n";}

};

class Derived2 : public Derived1
{
public:
    Derived2 () {cout << "Derived2::Derived2()\n";f();g();}
    Derived2 (Derived2 &) {cout <<
"Derived2::Derived2(Derived2&)\n";f();g();}

    void f () {cout << "Derived2::f()\n";g();}
    void g () {cout << "Derived2::g()\n";}

};

void fCopy (Base obj)
{
    obj.f ();
    obj.g ();
}

void main ()
{
```

```
Base *pBase = new Derived2;
```

a)

```
Derived1 *pDerived1 = new Derived2;
```

б)

```
Derived2 *pDerived2 = new Derived2;
```

в)

```
pBase->f ();
```

г)

```
pDerived1->f ();
```

д)

```
pDerived2->f ();
```

е)

```
fCopy (*pBase);
```

ж)

```
delete pBase, pDerived1, pDerived2;
```

```
}
```

**Задача 5.** (10 точки) Да се попълни в празните полета изходът от предшестващите ги програмни конструкции. Да се обозначи типът на свързването с адреси на методи – (д)инамично или (с)татично.

```
class A
{ public:
    A()
    { cout << "A::A()\n"; g(); 
    }
    void operator=(const A&)
    { cout << "A::=\n";
    }
    void f()
    { cout << "A::f()\n";
      g(); 
    }
    virtual void g ()
    { cout << "A::g()\n";
    }
    virtual ~A()
    { cout << "A::~~\n";
    }
};

class B : public A
{ public:
    B()
    { cout << "B::B()\n";
    }
    void f()
    { cout << "B::f()\n";
    }
    void g()
    { cout << "B::g()\n";
    }
    ~B()
    { cout << "B::~~\n";
    }
};
```

```
void main()
{ cout << "1:\n";
  B* bp = new B;
  

  cout << "2:\n";
  A* bp_a = bp;
  bp_a->f(); 
```

```
  cout << "3:\n";
  bp_a->g(); 
```

```
  cout << "4:\n";
  A a;
  a = *bp;
```

```
  cout << "5:\n";
  B b;
```

```
  b = *bp;
```

```
  cout << "6:\n";
  delete bp_a; 
```

```
  cout << "7:\n";
```

```
};
```

**Задача 6.** (15 точки) Да се реализира абстрактен клас **Function**, представящ целочислена функция на една променлива. Да се реализират негови наследници **LinearFunction**, **CharacteristicFunction** и **Superposition**, представящи съответно линейна функция, характеристична функция на крайно множество от цели числа и суперпозиция на две функции, представени чрез обекти от тип **Function**.

Линейната функция от вида  $f(x) = ax + b$  се задава чрез коефициентите  $a$  и  $b$ . Характеристичната функция на дадено множество е такава функция, която за елементите на множеството приема стойност 1, а за всички останали стойности на аргумента – 0. Задава се чрез самото множество, представено чрез динамичен масив. Суперпозиция на две функции  $f$  и  $g$  е такава функция  $h = f.g$ , за която  $h(x) = f(g(x))$  за всяко  $x$  и се задава се чрез (указатели към) два обекта от клас **Function**, представящи  $f$  и  $g$ .

За класовете да се реализират подходящи конструктори, метод за изчисляване на стойността на функцията и ако е необходимо – деструктори.

**Задача 6.** (10 точки) Да се попълни в празните полета изходът от предшестващите ги програмни конструкции.

```
#include <iostream.h>
class Base
{ public:
    Base()
    { cout << "Base::Base()\n";
    }
    Base(const Base&)
    { cout << "Base::copy()\n";
    }
    virtual void f()
    { cout << "Base::f()\n";
    }
    virtual ~Base()
    { cout << "Base::~~\n";
    }
};
class Der : public Base
{ public:
    Der()
    { cout << "Der::Der()\n";
    }
    Der(const Der&)
    { cout << "Der::Copy()\n";
    }
    void f()
    { cout << "Der::f()\n";
    }
    ~Der()
    { cout << "Der::~~\n";
    }
};
void g1(Base a)
{ cout << "F:";
  a.f();
}
void g2(Der a)
{ cout << "F:";
  a.f();
}
void g3(Base& a)
{ cout << "F:";
  a.f();
}
```

```
void main()
{ cout << "1:\n";
  Der d;
```

```
    cout << "2:\n";
    g1 (d);
```

```
    cout << "3:\n";
    g2 (d);
```

```
    cout << "4:\n";
    g3 (d);
```

```
    cout << "5:\n";
```

```
}
```

**Задача 7.** (15 точки) Да се реализира абстрактен клас **Sequence**, представящ безкрайна редица от числа и метод за достъп до нейните елементи (по пореден номер). Да се реализират класовете **ArithmeticProgression**, **PeriodicSequence** и **InterleavedSequence**, наследници на класа **Sequence**, които представят съответно аритметична прогресия, периодична редица и комбинация на две редици.

Периодична редица е такава безкрайна редица от числа, която се получава като последователно повторение на крайна редица от числа безкраен брой пъти. Комбинация на две редици  $a_1, a_2, \dots, a_n, \dots$  и  $b_1, b_2, \dots, b_n, \dots$  е редицата  $a_1, b_1, a_2, b_2, \dots, a_n, b_n, \dots$

За класовете да се реализират подходящи конструктори, мутатори (методи за промяна на стойността на полетата) и ако е необходимо – деструктори.

**Задача 5.** (10 т.) *Задачата да се реши на езика C++ или Java. В началото на вашето решение посочете кой език сте избрали.*

А) Да се дефинира структура `ChessPosition`, описваща коректна позиция на фигура върху шахматна дъска (координатите на позицията са от 'А' до 'Н' по едното измерение и от 1 до 8 по другото).

Да се дефинира абстрактен клас (или интерфейс) `ChessPiece`, описващ шахматна фигура със следните операции:

- `ChessPosition getPosition()` – Връща позицията на фигурата на дъската;
- `[подходящ тип] allowedMoves()` – Връща списък (колекция) с всички възможни позиции, до които дадена фигура може да достигне с един ход;
- `[булев тип] captures(ChessPosition pos)` – Проверява дали фигурата “владее” позицията `pos`, подадена като параметър, т.е. дали позицията е в списъка с възможните ходове на фигурата. Булевият тип да бъде булевият тип в езика, който сте избрали (напр. `bool`, ако пишете на C++).

Б) Да се дефинират класовете `Rook` и `Knight` – наследници на `ChessPiece`, описващи съответно шахматните фигури топ и кон.

В) „Стабилна конфигурация“ наричаме такава подредба на фигурите върху дъската, при която никоя фигура да не е върху позволен ход на друга фигура (т.е. никои две фигури да не се „бият“). Да се дефинира функцията `allMoves` (`[подходящ тип] pieces[, ...]`), която за списъка (колекцията) `pieces`, съдържащ произволен брой разнородни шахматни фигури, отпечатва на конзолата всеки възможен ход на фигура от `pieces` такъв, че след изпълнението му списъкът с фигури да описва стабилна конфигурация. Информацията за ходовете да съдържа типа на фигурата, старата позиция и новата позиция, например:

```
Rook A1 -> B1  
Knight B3 -> A5
```

*Забележка: Реализирайте всички конструктори и други операции, които смятате, че са необходими на съответните класове.*

**Задача 5.** (10 т.) *Задачата да се реши на езика C++ или Java. В началото на вашето решение посочете кой език сте избрали.*

Нека GameBoard е предварително дефинирана квадратна матрица от цели числа с размери  $N \times N$ , представляваща игрова дъска. Всеки елемент в матрицата има стойност 0 („земя”), 1 („огън”) или 2 („вода”). За две позиции в матрицата  $(i, j)$  и  $(i', j')$  казваме, че са съседни, ако  $|i - i'| \leq 1$  и  $|j - j'| \leq 1$ .

А) Да се дефинира структура Point, описваща позиция на игровата дъска. Да се дефинира абстрактен клас (или интерфейс) GamePlayer, който описва играч на игровата дъска със следните операции:

- `getPosition()` – Връща позицията на играча на дъската;
- `allowedMoves()` – Връща списък (колекция) с всички възможни позиции, до които играчът може да достигне с един ход.

Б-1) Да се дефинира клас Knight, наследник на GamePlayer, описващ „сухопътен рицар“. Рицарят може да се придвижва само в такава съседна позиция, която е „земя” и не е в съседство с „огън”. Пример за достижими позиции за рицаря К е показан на диаграмата вдясно.

|   |   |   |
|---|---|---|
| 1 | 0 | 1 |
| 0 | K | 2 |
| 0 | 0 | 2 |

Б-2) Да се дефинира клас SeaMonster, наследник на GamePlayer, описващ „морско чудовище“. Морското чудовище може да се придвижва с произволен брой позиции по хоризонтала или по вертикала, но само по „вода”. Пример за достижими позиции за чудовището S е показан на диаграмата вдясно.

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 1 | 0 | 2 | 0 |
| 0 | 2 | 1 | 0 | 2 |
| 2 | S | 2 | 2 | 1 |
| 1 | 1 | 2 | 2 | 0 |
| 2 | 2 | 1 | 1 | 1 |

В) „Война“ наричаме такава подредба на играчите по дъската, при която на някоя от съседните позиции на всеки играч има друг играч. Да се дефинира функция

`allMoves ([подходящ тип] players[, ...])`

която по даден списък (колекция) `players`, съдържащ произволен брой разнородни играчи, извежда на стандартния изход всеки възможен ход на играч от `players` такъв, че след изпълнението му списъкът с играчи да описва война. Информацията за ходовете да съдържа типа на играча, старата позиция и новата позиция.

#### Пример:

`Knight (0,0) -> (1,1)`

`SeaMonster (2,2) -> (5,2)`

*Забележка: реализирайте всички конструктори и други операции, които смятате, че са необходими на съответните класове.*

## Задача 2.

Задачата да се реши на езика C++. Дадени са дефинициите:

```
class A {
public:
    A() { cout << "A()\n"; }
    A(A&) { cout << "A(A&)\n"; }
    virtual ~A() { cout << "~A()\n"; }
    A& operator=(A&) {
        cout << "op=(A&)\n";
        return *this;
    }
};

class B : public A {
public:
    B() { cout << "B()\n"; }
    B(B&) { cout << "B(B&)\n"; }
    virtual ~B() { cout << "~B()\n"; }
    B& operator=(B&) {
        cout << "op=(B&)\n";
        return *this;
    }
};

void f(A b) { cout << "f(A)\n"; }
```

Под всеки от редовете на дадения вдясно программен фрагмент да се посочи какво ще се изведе в резултат от неговото изпълнение. (Между редовете нарочно е оставено повече място, за да може да попълните отговора си) Ако смятате, че някой ред няма да изведе нищо, напишете “не извежда нищо”. Ако смятате, че някой от редовете ще предизвика грешка, напишете “грешка” и обяснете каква е тя и защо възниква.

За коректни се считат отговорите, които напълно съответстват на това, което ще се случи за съответния ред. Текст “грешка” без обяснение носи нула точки.

```
B d;

B copy = d;

A b = d;

A& ref = d;

B arr[2];

f(d);

A* p = new B(d);

delete p;

d = d;

ref = d;
```

**Задача 2.** Задачата да се реши на езика C++.

1) Освен конструктора по подразбиране (default constructor), кои други функции влизат в “голямата четворка” (функциите от т.нар. “rule-of-3”)?  
Да се попълнят имената им в полетата долу:

---

---

---

2) Нека е дадена дефиницията:

```
class foo {  
public:  
    virtual void f() {};  
    void g() {};  
};
```

Срещу всеки от редовете, които извикват f или g, да се запише “статично” или “динамично” според вида свързване, който ще се използва за тях.

```
foo obj;  
foo& ref = obj;  
obj.f(); _____  
obj.g(); _____  
ref.f(); _____  
ref.g(); _____
```

3) Нека са дадени следните дефиниции:

```
class base {  
public: int a;  
private: int b;  
};  
class derived : protected base { };
```

Да се посочи каква ще бъде видимостта на променливите a и b в класа derived – public, protected или private.

- Видимост на a: \_\_\_\_\_
- Видимост на b: \_\_\_\_\_

4) Нека класът X е абстрактен. Срещу всяко от твърденията да се посочи “да” или “не” според това дали е вярно:

- Могат да се създават обекти от тип X: \_\_\_\_\_
- Могат да се създават референции (reference) към обекти от тип X: \_\_\_\_\_

5) Нека е дадена следната дефиниция:

```
struct s {  
public:  
    static int var;  
    s() { var = 5; }  
};  
int s::var = 0;
```

Да се посочи какво ще изведе следният фрагмент:

```
std::cout << '(' << s::var << ')';  
s obj1;  
obj1.var = 10;  
s obj2;  
std::cout << '-' << s::var << '-';
```

6) Да се допълни дефиницията на класа test, така че функцията f да бъде чиста виртуална (pure-virtual) и класът да може коректно да се използва като основа на полиморфна йерархия.

```
class test {  
public:  
    _____ void f() _____;  
};
```

7) Да се допълни дефиницията на шаблона Array, така че функцията test да се компилира коректно и да извежда на стандартния изход 55.

```
_____ <_____>  
class Array {  
    static const size_t size = 10;  
    T data[size];  
public:  
    _____ at(size_t index) {  
        if (index _____)  
            throw std::out_of_range("error");  
        return data[index];  
    }  
};
```

```
void test() {  
    Array<int> a;  
    a.at(0) = 5;  
    std::cout << a.at(0);  
    Array<Array<int>> b;  
    b.at(0) = a;  
    std::cout << b.at(0).at(0);  
}
```

**Задача 2.** Дадена е следната програмата на езика за програмиране C++, от която липсват части.

Класовете и шаблоните Inc, Square, Sum и Max описват едноместни функции от тип  $f : T \rightarrow T$ . Видът на конкретната функция се дефинира от метода value в съответния клас.

Класът Inc представя функцията  $f : \text{double} \rightarrow \text{double}$ ,  $f(x) = x + 1$ .

Класът Square представя функцията  $f : \text{double} \rightarrow \text{double}$ ,  $f(x) = x^2$ .

Шаблонът на клас Sum представя функцията  $f : T \rightarrow T$ ,  $f(x) = f_1(x) + \dots + f_k(x)$ , където  $f_1(x), \dots, f_k(x)$ ,  $k \geq 0$  е списък от функции от тип  $f_i : T \rightarrow T$ . Дадена функция се добавя към списъка с функции на обект от клас Sum<T> чрез метода addFunction.

Шаблонът на клас Max представя функцията  $f : T \rightarrow T$ ,  $f(x) = \max\{f_1(x), \dots, f_k(x)\}$ , където  $f_1(x), \dots, f_k(x)$ ,  $k > 0$  е списък от функции от тип  $f_i : T \rightarrow T$ . Дадена функция се добавя към списъка с функции на обект от клас Max<T> чрез метода addFunction.

Function е шаблон на абстрактен клас, който е базов за Inc, Square, Sum и Max.

Функцията main въвежда от стандартния вход числото  $x$  и извежда най-голямата измежду стойностите  $x + 1$ ,  $x^2$  и  $x^2 + x + 1$ .

Да се попълнят липсващите части в програмата. Да се приеме, че класовете Sum и Max не е нужно да правят копие на подадените им функции.

```
#include <vector>
#include <iostream>
template <typename T>
class Function
{ public:
    _____ T value(T) const _____
};
class Inc : _____
{ public:
    double value(double x) const { return x+1; }
};
class Square : _____
{ public:
    double value(double x) const { return x*x; }
};
```

```
_____

class Max : _____
{ private:
    std::vector<_____> functions;
public:
    void addFunction(_____ f)
    { functions.push_back(f); }
    T value(T x) const
    {
        if(_____ )
            throw "Function list is empty!";

        return _____;
    }
};
```

```
_____
};

_____

class Sum: _____
{ private:
    std::vector<_____> functions;
public:
    void addFunction(_____ f)
    { functions.push_back(f); }
    T value(T x) const
    {
        _____
        return _____;
    }
};

int main()
{
    Inc i; Square sq; Sum<double> s;
    //(x+1)+(x*x)
    s.addFunction(&i); s.addFunction(&sq);
    Max<double> m;
    //{x+1, x*x, (x+1)+(x*x)}
    m.addFunction(&i); m.addFunction(&sq);
    m.addFunction(&s);
    double x; std::cin >> x;
    std::cout << m.value(x) << std::endl;
}
```

**Задача 2.** Задачата да се реши на езика C++.

**Документ** наричаме символен низ с произволна дължина, който има име, също символен низ. Текстът в документа е съставен от редове, разделени със символа '\n'.

**Папка** наричаме контейнер, който може да съдържа както документи, така и други папки. Казваме, че subfolder е **вложена папка** във folder, ако subfolder е елемент на контейнера folder или е вложена в някоя папка, която е вложена във folder.

В следната програмата на езика за програмиране C++ липсват части.

Класът Document описва документ, а класът Folder описва папка. Класът File е абстрактен базов клас за Document и Folder, който дефинира операцията search за търсене на символен низ в йерархия от документи и папки.

**Резултат от търсенето** на низа str в даден документ doc наричаме такава тройка (name, N, line), където name е името на документа doc, line е съдържанието на някой ред в документа, съдържащ str като подниз, а N е поредния номер на line в документа.

Резултатът от doc.search(str) е вектор с всички резултати от търсенето на str в doc.

При търсене в папка folder, folder.search(str) е вектор с всички резултати от търсенето на str в документите, съдържащи се във folder или в нейните вложени папки.

Да се попълнят липсващите части в програмата. При правилно заместване на празните места, програмата ще изведе следното на стандартния изход:

```
employees.txt, line 1: John Smith
employees.txt, line 2: Jane Smith
```

Да се приеме, че класът Folder не е нужно да прави копие на вложените обекти. Можете да използвате всякакви библиотечни функции като допишете съответните #include директиви.

```
#include <vector>
#include <iostream>

struct SearchResult
{
    std::string fileName;
    unsigned line_number;
    std::string line;
};
class File
{
public:
    _____ std::vector<SearchResult> search(_____) const _____

    virtual ~File(){}
};
class Document: public File
{
public:
    std::string name;
    std::string contents;
    Document(_____ _name, _____ _contents):_____

    std::vector<SearchResult> search(_____ str) const
    {
        std::vector<SearchResult> result;
```

---

```
        return result;
    }
};
class Folder: public File
{
    _____ files;
    std::string name;
public:
    Folder(_____ _name):_____

    void addFile(_____ f)
    {files.push_back(f);}

    std::vector<SearchResult> search(_____ str) const
    {
        std::vector<SearchResult> result;

        _____

        return result;
    }
};
int main()
{
    Document d1("employees.txt","John Smith\nMaryia Ivanova\n"),
        d2("inventory.txt","Computers: 3\nPrinters: 1"),
        d3("employees.txt","Ivan Petrov\nJane Smith"),
        d4("inventory.txt","Computers: 5, 3D Printers: 1");
    Folder root("root"), acme("ACME Soft, Inc."), best("Best Soft, OOD");
    acme.addFile(&d1); acme.addFile(&d2);
    best.addFile(&d3); best.addFile(&d4);
    root.addFile(&acme); root.addFile(&best);
    std::vector<SearchResult> results = root.search("Smith");
    for(unsigned i = 0; i < results.size(); ++i)
    {
        std::cout << results[i].fileName
            << ", line " << results[i].line_number << ": "
            << results[i].line << std::endl;
    }
}
```

---

**Задача 2.** Задачата да се реши на езика C++.

Разгледайте дадения по-долу фрагмент. След това отговорете на въпросите, които следват.

```
1 class Base {
2 public:
3     ~Base() {}
4     virtual Base& self() { return *this; }
5 };
6 class Derived : virtual public Base {
7     int* ptr;
8 public:
9     Derived() { ptr = new int[10]; }
10    ~Derived() { delete[] ptr; }
11    Derived& self() { return *this; }
12 };
13 int main() {
14     Base* b = new Derived();
15     delete b;
16 }
```

1) Срещу всяко от твърденията напишете „Да“, ако то е вярно, или „Не“, ако е невярно.

|                                                                                    |  |
|------------------------------------------------------------------------------------|--|
| А) Класът Base е абстрактен, защото в него има виртуална функция.                  |  |
| Б) Класът Base е полиморфен, защото в него има виртуална функция.                  |  |
| В) Функцията Base::self е чисто виртуална (pure virtual).                          |  |
| Г) Функцията Derived::self е public, защото и Base::self е public.                 |  |
| Д) Base::self връща референция (reference) към временен обект.                     |  |
| Е) В Base::self има грешка, защото връща указател, а типът ѝ е референция – Base&. |  |

2) По-долу са описани няколко наблюдения върху по-горния фрагмент. Срещу всяко от тях напишете „Грешка“, ако то описва проблем в кода или „ОК“, ако описва валидна конструкция.

|                                                                                                                |  |
|----------------------------------------------------------------------------------------------------------------|--|
| А) Деструкторът на Base не е виртуален, а съдържа виртуална функция и наследникът му заделя динамични ресурси. |  |
| Б) Деструкторът на Base е публичен.                                                                            |  |
| В) self е дефинирана като virtual в Base, но не и в Derived.                                                   |  |
| Г) Типът на Base::self е Base, но Derived::self има тип Derived.                                               |  |

3) Разгледайте дадения по-долу фрагмент.

```
1 virtual class Foo {
2 public:
3     virtual Foo() {}
4     virtual ~Foo() {}
5     virtual int var;
6     int func(virtual int param) {
7         return param;
8     }
9     virtual void anotherFunc() {
10         virtual int localVar;
11     }
12 };
13
14 class Bar : virtual public Foo { };
```

На кои редове запазената дума virtual е използвана коректно? В полетата за отговор по-долу напишете „Да“ срещу всяко място, където употребата е коректна и „Не“, където има грешка.

|                                                     |  |
|-----------------------------------------------------|--|
| ред 1, дефиниция на клас                            |  |
| ред 3, дефиниция на конструктор                     |  |
| ред 4, дефиниция на деструктор                      |  |
| ред 5, дефиниция на член-променлива                 |  |
| ред 6, дефиниция на параметър на функция            |  |
| ред 9, дефиниция на член-функция                    |  |
| ред 10, дефиниция на локална променлива във функция |  |
| ред 14, наследяване на клас                         |  |

4) Довършете програмата на подчертаните места, така че да извежда текст „АВС“ на екрана.

```
1 // Обърнете внимание, че функциите
2 // извеждат точно по един символ!
3 class A {
4 public:
5     A() { std::cout << '___'; }
6     _____ void f() { std::cout << '___'; }
7 };
8 class B : public A {
9 public:
10    B() { std::cout << '___'; }
11 };
12 int main() {
13     ___ obj;
14     ___::f();
15 }
```

**Задача 2.** Задачата да се реши на езика C++.

1) Разгледайте дадената по-долу програма. В полето за отговор напишете какво ще изведе тя.

```
1 class A {
2 public:
3     A() { prn('c'); }
4     ~A() { prn('d'); }
5     virtual void prn(char ch) const {
6         std::cout << 'A' << ch << ' ';
7     }
8 };
9
10 class B : public A {
11 public:
12     B() { prn('c'); }
13     ~B() { prn('d'); }
14     virtual void prn(char ch) const {
15         std::cout << 'B' << ch << ' ';
16     }
17 };
18
19 int main() {
20     A* obj = new B;
21     obj->prn('p');
22     delete obj;
23 }
```

Отговор: \_\_\_\_\_

2) Срещу всяко от твърденията напишете „Да“, ако то е вярно, или „Не“, ако е невярно.

|                                                                                                                                          |  |
|------------------------------------------------------------------------------------------------------------------------------------------|--|
| А) Конструкторът на един клас може да бъде статичен.                                                                                     |  |
| Б) Деструкторът на един клас може да бъде статичен.                                                                                      |  |
| В) В статична член-функция на даден клас можем да достъпваме както статичните, така и нестатичните член-променливи на класа.             |  |
| Г) В статична член-функция на даден клас можем да направим обръщение към виртуална член-функция на същия клас без да има създаден обект. |  |
| Д) Във виртуална функция на даден клас можем да направим обръщение към статична член-функция на същия клас.                              |  |
| Е) Ако в един абстрактен клас има член-функция, тя задължително трябва да бъде виртуална.                                                |  |

3) Довършете програмата (на подчертаните места) така, че да бъде синтактично коректна и да спазва принципите на ООП в C++.

```
1 class Item {
2     std::string m_label;
3 public:
4     Item(std::string t) : m_label(t) {}
5
6     _____ ~Item() {}
7
8     const std::string& label() const {
9         return m_label;
10    }
11 };
12
13 _____ <typename T>
14 class Box : public Item{
15     _____ m_contents;
16 public:
17     Box(std::string label, const T& contents)
18         : Item(_____){
19         {
20             m_contents = new T(_____);
21         }
22
23     Box(const Box&) = delete;
24     Box& operator=(const Box&) = delete;
25     _____ ~Box() { delete m_contents; }
26     T& contents() { return _____; }
27 };
28
29 Box<int>* createRandomIntBox() {
30     int randomValue = rand() % 100 + 1;
31     return _____("
32         random box", randomValue);
33 }
34
35 Box<int>* toIntBox(Item* p) {
36     return dynamic_cast<Box<int>*>(p);
37 }
38
39 int main() {
40     Item* p = createRandomIntBox();
41     toIntBox(p)->contents() = 42;
42     std::cout << "Box {"
43         << "\n Label: " << p->label()
44         << "\n Contents: "
45         << toIntBox(p)->contents()
46         << "\n\n";
47     delete p;
48 }
```